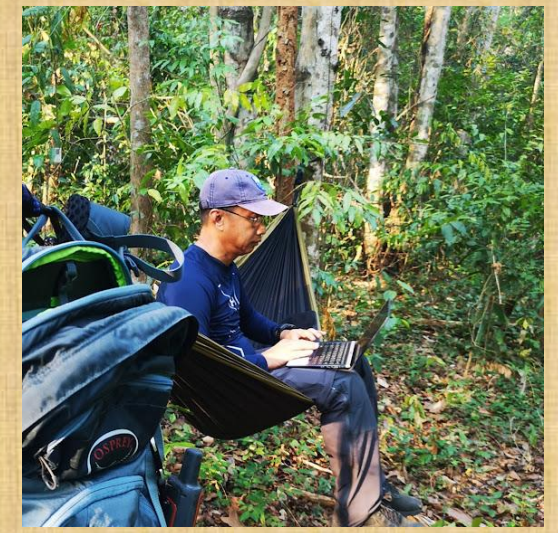
ITCS214 : DATA STRUCTURES & ALGORITHM ANALYSIS



Lecture 3 · Array-Based Structures



From Fixed Arrays to Dynamic Arrays and Amortized Analysis

Asst. Prof. Dr. Ananta Srisuphab
ananta.sri@mahidol.ac.th

2026/1

Mahidol University
Faculty of Information and Communication Technology

BY THE END OF TODAY

Learning Outcomes

01

Arrays & Indexing

- Explain indexed access as O(1)
- Distinguish logical size from physical capacity
- Explain why a fixed array can't grow

02

Build It Yourself

- Implement an array-backed list
- Detect and handle a full backing array
- Implement grow() / ensureCapacity()

03

Growth & Amortized Analysis

- Compare array-growth strategies
- Derive $\Theta(n^2)$ for grow-by-one
- Derive $\Theta(n)$ → amortized O(1) for doubling

04

Complexity & Verification

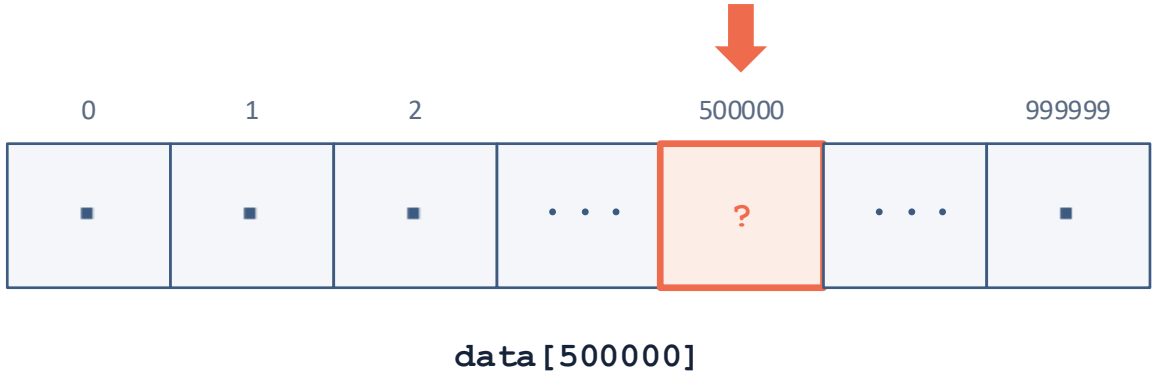
- Analyze complexity of common operations
- Inspect AI-generated code for correctness AND performance

Does Indexed Access Scan the Array?

PREDICT BEFORE WE RUN IT

```
int x = data[i];
```

If data contains one million elements, does accessing data[500000] require checking the previous 499,999 elements first?



One jump straight to the address — no scanning through index 0…499999.

Three Growth Rates We'll Use All Lecture

O(1)

CONSTANT

Same cost no matter how large the input.



O(n)

LINEAR

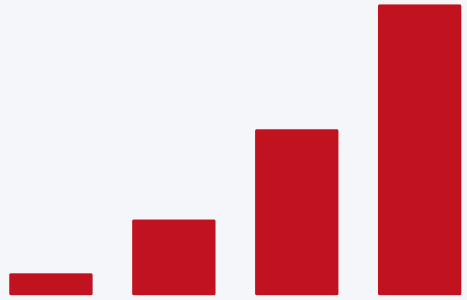
Cost grows proportionally with input size.



O(n^2)

QUADRATIC

Cost grows with the square of input size.



We Don't Know n in Advance

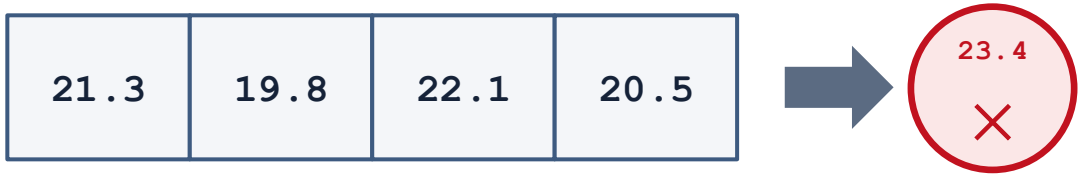
Sensor readings arrive continuously. We don't know how many will arrive.

An array gives excellent indexed access — but its length is fixed at creation.

CENTRAL QUESTION

How can we build a list that gives us array-like indexed access, but can grow when we need more space?

```
readings = new double[4];
```



`size = 4` `capacity = 4` — `no room for the 5th reading`

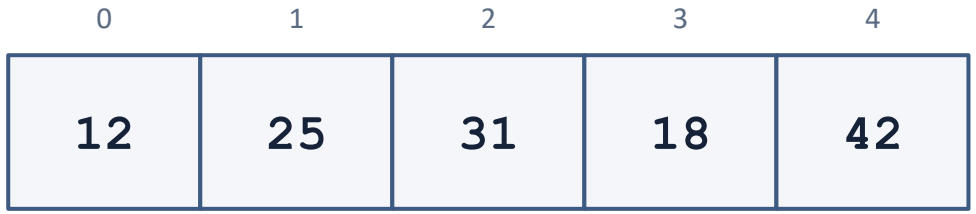
What we actually want:

- Array-style get(i) in O(1)
- No fixed limit chosen up front
- Grows only when it truly needs to

Java Arrays: Fixed-Length, Indexed Storage

```
int[] data = new int[5];

int n = 10;
int[] data2 = new int[n];
```



What an array gives us

- Length is fixed once the array is created
- Valid indices run from 0 to length − 1
- data[i] is O(1) — direct, not sequential
- Can hold primitive values or object references

THE CATCH

Once new int[5] runs, that length is locked in for the life of the array.

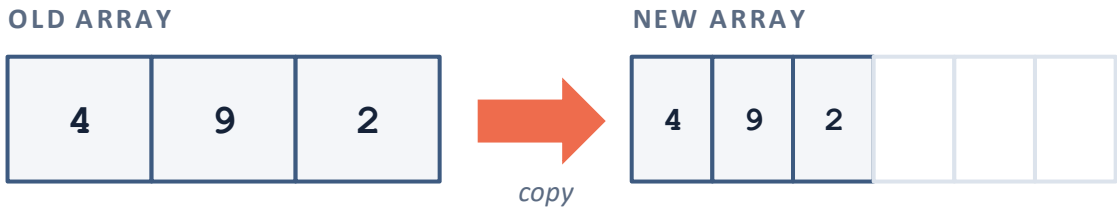
A Fixed Array Cannot Grow Itself

There is no resize() method on an array. The only way to get more room is to create an entirely new, bigger array and copy everything over.

```
int[] bigger = new int[data.length * 2];
System.arraycopy(
    data, 0, bigger, 0, data.length
);
data = bigger;
```

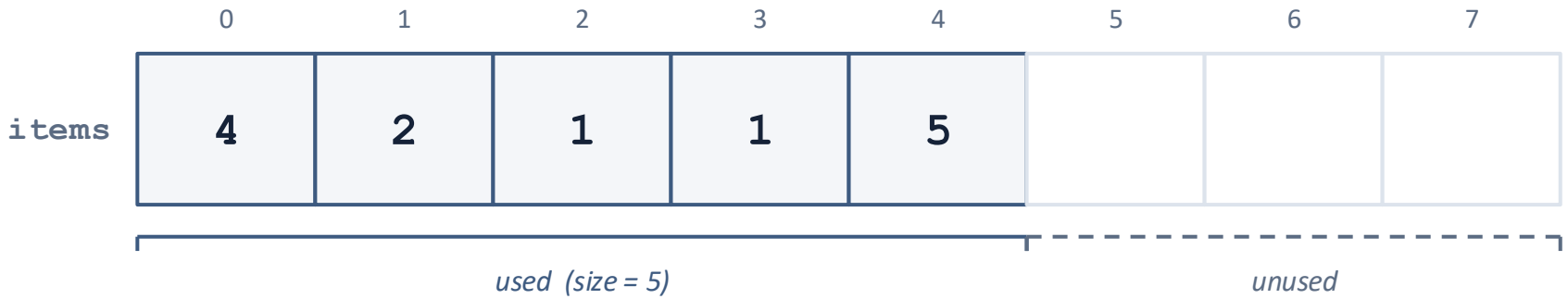
THE REAL COST

Doing this by hand, every single time, is tedious and error-prone. We're about to automate it inside our own class.



Every element must be copied — this costs time proportional to how many elements exist.

Size vs. Capacity




```
0 <= size <= items.length
```

size
Number of logical elements the list currently holds — what the user thinks of as "the list."

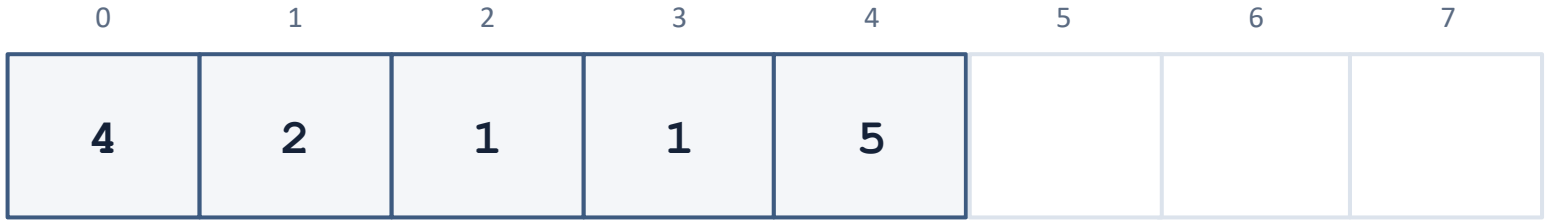
capacity
items.length — total room in the backing array, whether or not it's all in use.

PREDICT BEFORE WE RUN IT

Predict: Where Does addLast() Write?

`addLast(x)` — *appends x as the new final element of the list. (We'll write the real Java for this soon — for now, just the idea.)*

If size == 5 and items.length == 8, what index will the next addLast() use?



THE BIG IDEA

A Dynamic Array = Three Simple Ingredients

1

A backing array

Plain fixed-length array that actually stores the elements.

```
int[] items;
```

2

A size counter

Tracks how many elements are logically in use.

```
int size;
```

3

A resize step

Swaps in a bigger backing array exactly when needed.

```
if (size == items.length)
  grow();
```

KEY IDEA

None of this is magic — it's an array, a counter, and a rule for when to replace the array. That's the entire mechanism.

ArrayList: The Idea, Packaged

```
List<String> names = new ArrayList<>();
names.add("A");
names.add("B");
names.get(0);
```

- No size given up front — it grows as you add()
- Internally, ArrayList keeps a backing array
- When that array fills up, it's replaced with a bigger one automatically

> **Instead of treating ArrayList as magic, we are going to build the central mechanism ourselves.**
>
> *— today's real goal*

ArrayIntList: The Starting Skeleton

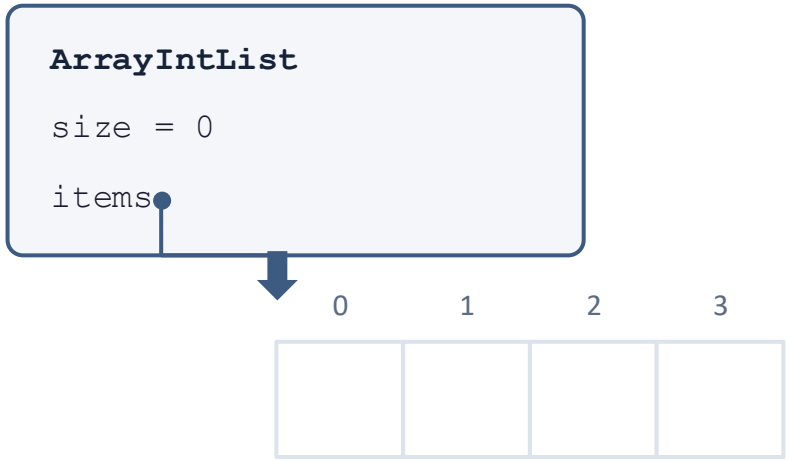
```
public class ArrayIntList {
    private int[] items;
    private int size;

    public ArrayIntList() {
        items = new int[4];
        size = 0;
    }
}
```

DESIGN CHOICE

We start with a small capacity (4) on purpose — so we hit the resizing case quickly and can watch it happen live.

new ArrayIntList()



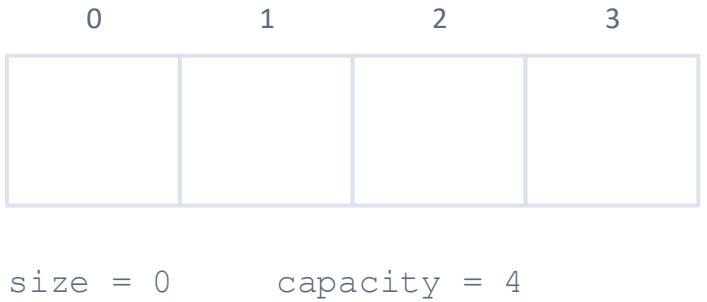

```
0 <= size <= items.length
```

```
valid elements:  items[0] ... items[size-1]
```

Representation invariant — must hold before and after every method call.

addLast(int data) — First Attempt

```
public void addLast(int data) {
    items[size] = data;
    size++;
}
```



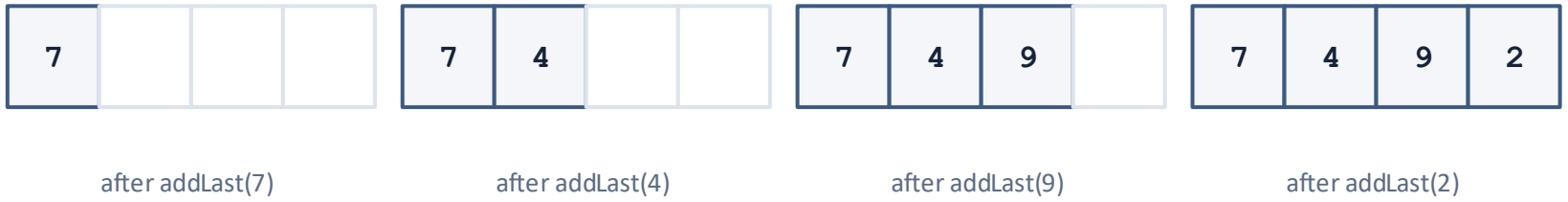
- Write the new value at index size — the next free slot
- Then increase size by one

LOOKS RIGHT...

It compiles, it works for a while — but we haven't asked what happens when the array is full.

Four Calls to addLast()

```
addLast(7);
addLast(4);
addLast(9);
addLast(2);
```



NOTICE

Every call so far writes into an empty slot that already exists. Nothing has been tested yet at the boundary.

PREDICT BEFORE WE RUN IT

What happens if we call addLast(8) now?

The Array Is Full



`size = 4` `capacity = 4`

```
addLast(8);
```

ArrayIndexOutOfBoundsException

`Index 4 out of bounds for length 4`

items[size] = items[4] doesn't exist — the backing array only has indices 0–3.

THE LESSON

Our naive addLast() assumed there would always be room. That assumption just broke — and every dynamic array must handle this moment.

WHAT MUST HAPPEN INSTEAD

Before writing, addLast() must first check: is the array full? If so, get a bigger one — before inserting.

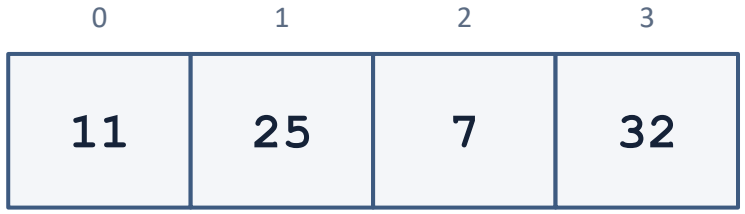
The Resizing Algorithm, Step by Step

1. Detect that the array is full (size == items.length)
2. Allocate a larger array
3. Copy the existing elements into it
4. Redirect items to point at the new array
5. Insert the new element — now there's room

Resizing — Allocate the New Array

STEP 1 The old array is full — nothing more fits.

old array (items)



`size = 4` `capacity = 4`

STEP 2 Allocate a brand-new, larger array — completely empty.

new array (expandedItems)

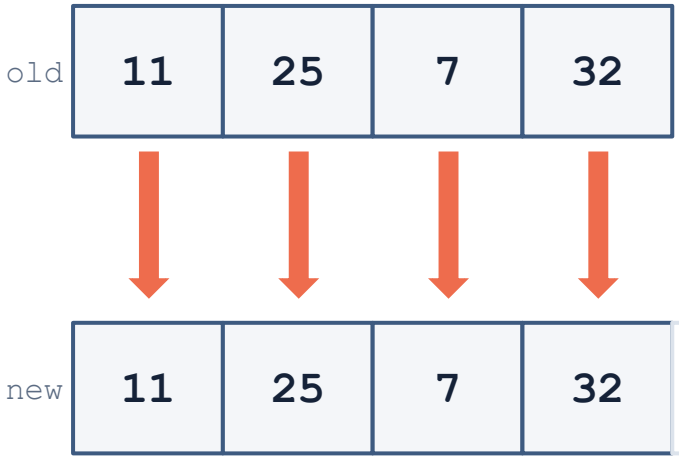


`capacity = 8`

```
int[] expandedItems =
    new int[newCapacity];
```

Resizing — Copy, Then Redirect

STEP 3 Copy every existing element into the new array.




```
System.arraycopy(items, 0, expandedItems, 0
, size);
```

STEP 4 Redirect items to the new array.

```
items = expandedItems;
```

Final state: size = 4, capacity = 8

The old array still exists in memory momentarily — but nothing refers to it anymore, so it becomes eligible for garbage collection.

addLast(), Fixed

```
public void addLast(int data) {
    if (size == items.length) {
        grow();
    }
    items[size] = data;
    size++;
}
```

Ordinary insertion

O(1)

size < items.length — just write and increment.

Insertion that resizes

O(n)

size == items.length — grow() must copy every element before the new one can be written.

LIVE-CODING CHECKPOINT 6

Test several insertions — enough to force at least one resize — and print size/capacity after each.

THE CENTRAL ALGORITHM-ANALYSIS QUESTION

How large should the new array be?

STRATEGY A

```
capacity = capacity + 1;
```

Grow by one element at a time

1 → 2 → 3 → 4 → 5 → … → n

STRATEGY B

```
capacity = capacity * 2;
```

Double the capacity each time

1 → 2 → 4 → 8 → 16 → … → 2^k

We won't just guess which is better — we'll derive it.

Why Growing by One Is Slow

Capacity sequence:

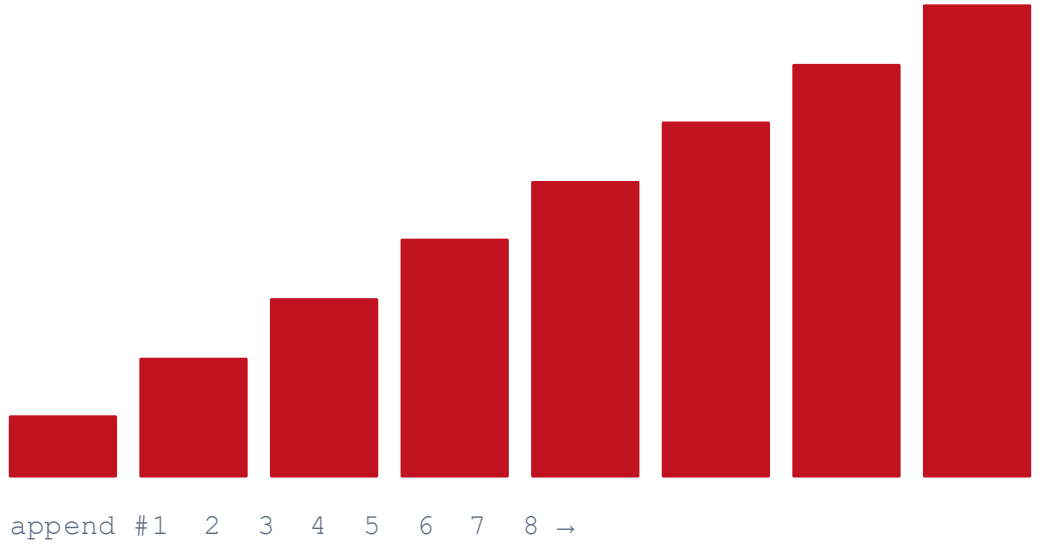
```
1 → 2 → 3 → 4 → 5 → ... → n
```

To append n elements, every single append after the first triggers a resize, so total copies are approximately:

```
1 + 2 + 3 + ... + (n - 1)
```

$$\frac{n(n-1)}{2} = T(n) = \Theta(n^2)$$

Copy cost at each resize event



Each bar's height = elements copied at that resize. Total area of all bars = the triangular sum above — quadratic in n.

CONCLUSION

Growing by one works correctly — but performs poorly when many elements are appended.

Why Doubling Is Fast, in Total

Capacity sequence:

```
1 → 2 → 4 → 8 → 16 → ... → 2^k
```

If n ≈ 2^k, the copying costs at each resize are:

```
1 + 2 + 4 + 8 + ... + n/2   <   n
```

(equivalently: 1 + 2 + 4 + ⋯ < 2n)

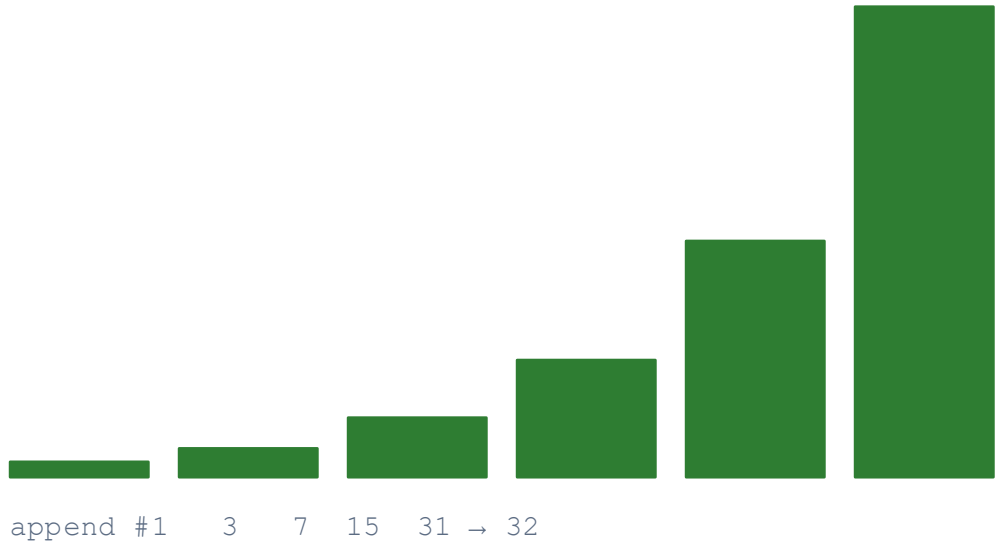
Total resize work

Θ(n)

Per append, on average

$$\frac{\Theta(n)}{n} = \Theta(1)$$

Copy cost at each resize event



Nearly all the total work is done by the LAST resize — and it alone is bounded by n. Earlier resizes are comparatively tiny.

CONCLUSION

Doubling keeps total copying work linear in n — spreading the cost so thinly that each append is, on average, constant time.

A Real Trace: 6 Appends, Starting from Capacity 1

Append #	size before → after	capacity before → after	Resize?	Elements copied
1	0 → 1	1 → 1	no	0
2	1 → 2	1 → 2	YES	1
3	2 → 3	2 → 4	YES	2
4	3 → 4	4 → 4	no	0
5	4 → 5	4 → 8	YES	4
6	5 → 6	8 → 8	no	0

Total elements copied

1 + 2 + 4 = 7

Bound check: is 7 < 2n?

7 < 2(6) = 12 ✓

Average copy cost / append

7 / 6 ≈ 1.17

Small n, same pattern: three of six appends resize, but the copying stays bounded by a small constant multiple of n — exactly what Θ(n) total work predicts.

NAMING THE IDEA

Amortized O(1) Append

One ordinary append

O(1)

One append that resizes

O(n)

addLast(), averaged over many calls

O(1) amortized

THE ANALOGY

Most operations are cheap; occasionally one expensive operation runs — but it "pays off" by making room for many future cheap ones. Spread that one expensive step over all the cheap steps it enabled, and the average per step is constant.

NOT the same as "average case"

Average-case analysis assumes a probability distribution over inputs. Amortized analysis makes no such assumption — it bounds the total cost over any sequence of n operations, guaranteeing the average per operation in that sequence, no probability required.

Discuss: Why Not Just “Average Case”?

DISCUSS WITH A PARTNER

Explain to each other, out loud, why amortized analysis does NOT require assuming a probability distribution over inputs — unlike average-case analysis.

Discussion prompts

- What guarantee does amortized O(1) actually give you, over a sequence of n operations?
- Could an adversary who chooses the worst possible sequence of operations break an amortized bound? Why or why not?
- Write ONE sentence, in your own words, defining “amortized cost.”

5 MINUTES — THEN WE'LL COLD-CALL A FEW PAIRS

What Java's ArrayList Actually Promises

Our teaching implementation

Doubles capacity every time it resizes, because it gives us clean, simple mathematics to derive by hand.

This is OUR design choice

Java's real java.util.ArrayList

Guarantees amortized constant-time append as a behavioral contract — it does NOT guarantee any particular growth factor in its public API.

Application code should not rely on it

IF YOU LOOK INSIDE OPENJDK'S SOURCE

You'll find it currently grows by roughly 1.5×, not 2×. That's an implementation detail of one particular JDK version — not part of the API guarantee, and it could change.

Generalizing: Any Growth Factor k > 1 Works

```
1 → k → k² → k³ → ... → k^m  ≈  n
```

Total copying work is a geometric series:

```
1 + k + k² + ... + k^(m-
```

k/(k-1) is just a constant (k is fixed), so:

Total work = Θ(n) → amortized cost = Θ(1)

for ANY fixed k > 1 — not just k = 2.

So why not pick k close to 1? Smaller k means more frequent (but individually cheaper) resizes:

Growth factor	k/(k−1)	Memory tradeoff
k = 1.5	3.0	Less wasted memory
k = 2.0	2.0	Balanced (our teaching choice)
k = 3.0	1.5	More wasted memory

CONNECTS BACK TO OPENJDK

This is exactly why OpenJDK can safely use ~1.5× instead of 2× — it still gets amortized O(1), just with a different constant, in exchange for wasting less memory after each resize.

Complexity of Every Common Operation

Operation	Complexity	Why
`size()`	**O(1)**	Stored field, direct read
`get(i)`	**O(1)**	Direct indexing
`set(i, x)`	**O(1)**	Direct indexing
`addLast(x)`	**O(1) amortized**	Occasional resizing, spread over many calls
`add(i, x)`	**O(n)**	Shifts elements to make room
`removeLast()`	**O(1)**	Just decrement size
`remove(i)`	**O(n)**	Shifts elements to close the gap
`indexOf(x)`	**O(n)**	Linear search, no shortcuts

Inserting in the Middle: add(2, 99)



In the worst case — inserting at index 0 — how many elements must move?

Every single one of them — all n elements shift one slot to the right.

add(i, x) → O(n)

Best case — inserting at the end — moves zero elements. That's exactly addLast().

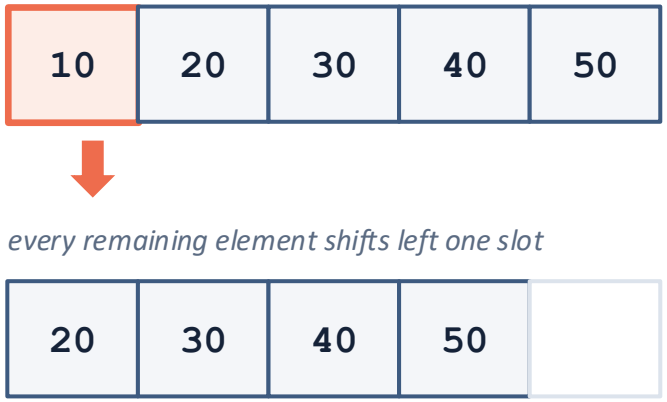
Removing Elements

```
public void removeLast() {
    if (size == 0) {
        throw new IllegalStateException(
            "List is empty");
    }
    size--;
    items[size] = 0;
}
```

WHAT ACTUALLY REMOVES THE ELEMENT

Decrementing size is what removes it logically — items[size] no longer counts as part of the list. Clearing the old value to 0 is just tidy-up, not the removal itself.

`remove(0)`



`removeLast()`

O(1)

`remove(0)`

O(n)

For object-backed lists, clearing the freed slot to null also prevents the array from unnecessarily holding a reference to a removed object.

Code Review: Is This AI-Generated addLast() Good?

```
public void addLast(int data) {
    if (size == items.length) {
        int[] bigger =
            new int[items.length + 1];
        System.arraycopy(
            items, 0,
            bigger, 0,
            items.length);
        items = bigger;
    }
    items[size++] = data;
}
```

The code appears correct. Is it a good implementation?

- Correctness
- Scalability
- Cost of n appends

Code Review #2: Is This remove(i) Safe?

```
public int remove(int i) {
    checkIndex(i);
    int removed = items[i];
    for (int j = i; j < size; j++) {
        items[j] = items[j + 1];
    }
    size--;
    return removed;
}
```

This passes every test the AI ran. Does it always work?

Typical case (i, size < capacity)

Full list (size == capacity)

Root cause

KEY TAKEAWAYS

From Fixed Arrays to Amortized O(1)

Fixed array

- Fast indexed access
- Fixed capacity — cannot grow

Dynamic array

- Backing array + size + resizing
- Grows by allocating & copying

Complexity

`get(i) / set(i,x)`	**O(1)**
`addLast(x)`	**O(1) am.**
`insert / remove (i)`	**O(n)**
`search`	**O(n)**

“Data structures are not just APIs. Their implementation choices determine their performance.”

EXTENSION TOPICS · OPTIONAL ENRICHMENT

Beyond ArrayIntList

Self-study before next lecture.

From ArrayIntList to MyArrayList<T>

ArrayIntList only stores int. A generic MyArrayList<T> stores any reference type — Strings, custom objects, anything — using the exact same design.

```
public class MyArrayList<T> {
    private T[] data;
    private int size;

    public MyArrayList(int capacity) {
        data = (T[]) new Object[capacity];
        size = 0;
    }
}
```

size, grow(), and the resizing algorithm are IDENTICAL to ArrayIntList — only the element type changes.

PREDICT BEFORE WE RUN IT

What error do you get if you write this directly?

```
T[] data = new T[capacity];
```

ANSWER

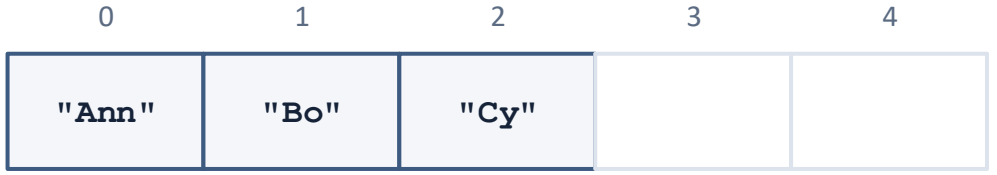
Illegal generic array creation. Java erases T at runtime, so it can't verify the array's real element type. The standard workaround: allocate Object[] and cast to T[], accepting one unchecked warning.

Generic Methods: addLast(T) and get(i)

```java
public void addLast(T value) {
    if (size == data.length) {
        grow();
    }
    data[size] = value;
    size++;
}

public T get(int i) {
    checkIndex(i);
    return data[i];
}
```

MyArrayList<String>



size = 3 capacity = 5

SAME BIG-O STORY

addLast(T x) triggers the exact same grow() / doubling logic you already derived: Θ(n) total copying work, O(1) amortized per call — regardless of what T is.

Enabling the For-Each Loop

```
public class MyArrayList<T>
        implements Iterable<T> {
    public Iterator<T> iterator() {
        return new Iterator<T>() {
            private int cursor = 0;
            public boolean hasNext() {
                return cursor < size;
            }
            public T next() {
                if (!hasNext())
                    throw new NoSuchElementException();
                return data[cursor++];
            }
        };
    }
}
```

hasNext() and next() let external code walk the list one element at a time without ever touching data or size directly — encapsulation preserved.

PREDICT BEFORE WE RUN IT

What does this actually compile to?

```
for (T item : list) {
    // use item
}
```

ANSWER

It's syntactic sugar. The compiler rewrites it into exactly this iterator pattern — call iterator() once, then loop on hasNext() / next().

```
Iterator<T> it = list.iterator();
while (it.hasNext()) {
    T item = it.next();
}
```

ArrayList vs. LinkedList: A Preview

Operation	ArrayList (ours)	LinkedList
`get(i)`	O(1)	O(n)
`addLast(x)`	O(1) amortized	O(1)
`addFirst(x)`	O(n) — shifts everything	O(1)
`add(i, x)  (middle)`	O(n)	O(n) to find + O(1) to link
`Memory per element`	Low (contiguous)	Higher (node + 2 references)
`Cache locality`	Good (contiguous)	Poor (scattered nodes)

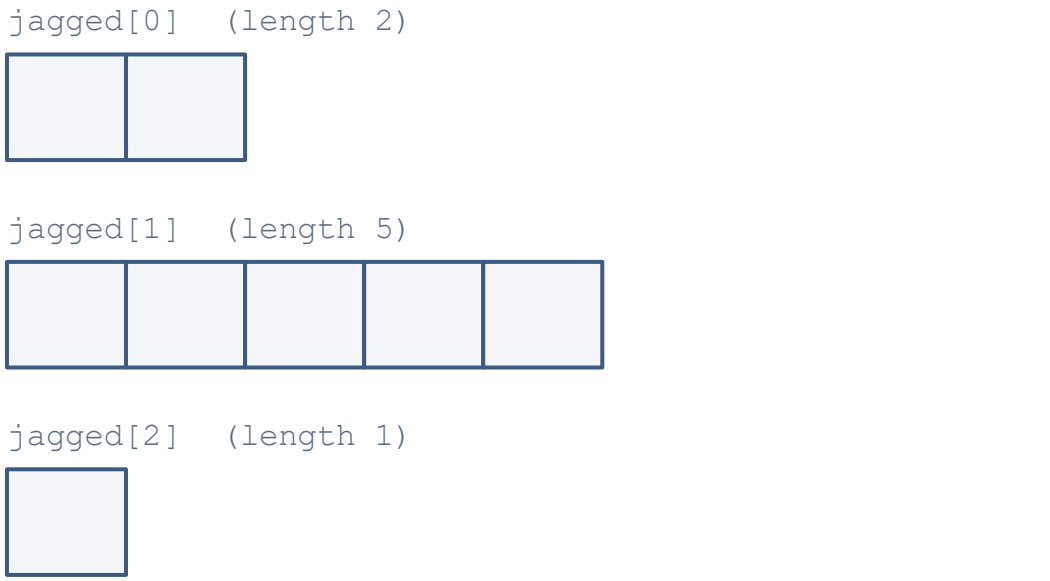
NO UNIVERSAL WINNER

The right structure depends on which operation dominates your workload. Next lecture: how a doubly-linked node structure achieves O(1) addFirst by giving up indexed access entirely.

2D and Jagged Arrays

```
int[][] grid = new int[3][4];
// rectangular: 3 rows x 4 cols

int[][] jagged = new int[3][];
// jagged: rows can differ in length
jagged[0] = new int[2];
jagged[1] = new int[5];
jagged[2] = new int[1];
```



KEY INSIGHT

In Java, a 2D array is really an array of array references — not one contiguous rectangular block. That's exactly why row lengths are allowed to differ.

Each row is its own independently-sized array object — nothing forces them to match.

grid[i][j] is two O(1) indexed lookups back to back: grid[i] returns row i's array reference, then [j] indexes into it. Still O(1) overall.